



Smart Punch-In System

Project Engineering

Year 4

Chuka Orajuba

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Chuka Orajuba

Acknowledgements

I would like to acknowledge and thank my project supervisor Michelle Lynch for her guidance and support throughout the project by helping me with any queries I wasn't too sure of and helping me with by giving me feedback each week on my project updates and demonstrations. I would also like to acknowledge my classmates who were using similar hardware and software for various parts of their project that helped me issues I ran into on the Raspberry Pi and helping me with how certain libraries work with the Raspberry Pi.

Table of Contents

1	Summary	6
2	Poster	7
3	Introduction	8
4	Background	9
4.1	Facial Recognition	9
4.2	Liveness Detection	9
4.3	FastAPI	9
4.4	Firebase Authentication.....	10
4.5	React and Next.js	10
4.6	SQLite	10
4.7	Server Sent Events	10
5	Project Architecture.....	11
6	Project Plan	12
6.1	Research.....	12
6.2	Hardware Setup	12
6.3	Backend Development.....	12
6.4	Frontend Development.....	12
6.5	Integration & Testing	13
7	Implementation	14
7.1	Face Recognition Pipeline	14
7.2	Liveness Detection	15
7.3	FastAPI Backend.....	16
7.4	Database Schema.....	17

7.5	Shift Roster System	17
7.6	Attendance Report.....	18
7.7	Frontend.....	19
8	Challenges and solutions	20
8.1	Wrong Operating system	20
8.2	SQLite Database Locking.....	20
8.3	Liveness Detection EAR Calibration	20
9	Results	21
10	Ethics	23
11	Conclusion.....	24
12	Appendix	25
12.1	Project File Structure	25
12.2	API Endpoint Reference	25
13	References	27

1 Summary

This project presents the design and implementation of a Facial Recognition Punch-In System which is built on a Raspberry Pi 4. The goal was to create a fully automated, contactless punch-in system that replaces the need for manual punch in system with real time facial identification. The scope of the project covers the full system stack, from the hardware assembly of the Raspberry Pi and the Raspberry Pi camera, with REST API backend with local database storage, to the webpage that is accessible by both employee and administrators. The project is designed to operate on a local network without the reliance on cloud infrastructure.

The system uses a Raspberry Pi Camera module and the `face_recognition` Python library to identify employees in real time. Once there is a face recognised, a liveness detection check based on Eye Aspect Ratio (EAR) blink analysis using the `dlib` library. The liveness check is added to the facial recognition to prevent photo spoofing before a clock event is recorded. All attendance data is stored locally in a SQLite database running on the Pi itself, keeping the system self-contained without requiring the need for external servers

The Raspberry Pi runs a FastAPI backend which serves a REST API and Server Sent Events (SSE) stream over the local network. A React and Next.js webpage provides employees with a live view of their attendance status, clock history, and upcoming shift schedule. On the other side the webpage provides an admin interface gives administrators full oversight including live punch-in status for all employees, a weekly shift roster builder, attendance punctuality report, and an employee linking system integrated with Firebase Authentication to register employees with their login credentials for the webpage.

I developed this project across two machines. The Raspberry Pi 4 was used for the backend and facial recognition and a laptop for the webpage frontend. The system was successfully demonstrated end to end with 100% recognition accuracy during testing combined with the reliable liveness detection that blocked all photo spoofing attempts, and very quick API response times on the local network. The main limitation encountered was that the recognition performance is directly dependent on the size of the training dataset, requiring a sufficient number of images per employee for consistent results.

2 Poster



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Smart Punch-In System

Chuka Orajuba
BEng Software & Electronic Engineering

PROJECT SUMMARY

This project presents a fully integrated, biometric attendance management system built on a Raspberry Pi 4. The system uses a camera module and the face_recognition library to identify employees in real time, automatically logging clock-in and clock-out events via REST API. All attendance records, employee profiles, shift rosters, and time entries are stored locally in a SQLite database running directly on the Pi, keeping the system self contained without requiring an external server. A React web dashboard provides employees and administrators with live attendance data, shift schedules, and attendance reports.

HARDWARE

- Raspberry Pi 4 Model B (4GB RAM)
- Raspberry Pi Camera Module v2
- Local SQLite database (on Raspberry Pi)
- Connected via home WiFi network
- Runs FastAPI backend and face recognition script simultaneously

SOFTWARE & LIBRARIES

BACKEND

- FastAPI (Python)
- SQLite Database
- Firebase Admin SDK
- Uvicorn ASGI Server
- face_recognition

RECOGNITION

- OpenCV
- Picamera2
- dlib (liveness detection)
- face_recognition lib
- HOG face detection

FRONTEND

- React / Next.js
- Firebase Auth
- Server-Sent Events
- JavaScript

SECURITY

- Firebase JWT tokens
- Device API key
- CORS protection
- secrets.compare_digest

BLOCK ARCHITECTURE

```

graph TD
    A[PI Camera Module  
Picamera2 • OpenCV • Raspberry Pi 4] -- Captures frame --> B[Face Recognition  
face_recognition library • HOG detection • 128D encodings]
    B -- Identifies face encoding --> C[Liveness Detection  
dlib landmark detector • EAR blink analysis • anti-spoofing]
    C -- Confirms liveness via blinking --> D[FastAPI Backend  
REST API • Server-Sent Events • Device key auth]
    D -- POST /clock/toggle --> E[SQLite + Firebase  
attendance.db on the Raspberry Pi • Firebase Auth JWT verification]
    E -- Stores event & verifies token --> F[React Dashboard  
Next.js App Router • Employee & Admin views • Mobile responsive]
  
```

KEY FEATURES

Real-Time Face Recognition
Identifies employees via Pi Camera using HOG detection and 128D face encodings. Supports multiple faces simultaneously with a 10 second cooldown per person logic.

Liveness Detection
Prevents photo spoofing using Eye Aspect Ratio (EAR) blink detection via dlib. Clock events only fire after a confirmed natural blink is detected.

Live Dashboard Updates
Server-Sent Events push clock-in/out events to all connected browsers instantly.

Admin Roster & Scheduling
Admin can build weekly shift rosters, assign shifts per employee per day, and view a full weekly calendar grid with colour coded shift types.

Attendance Report
Compares actual clock times against scheduled shifts. Automatically flags late arrivals, early departures, and no-shows with a configurable grace period.

Employee Registration
Admin selects a Pi-recognised employee from a dropdown, enters their email, and sends a Firebase password invite linking their face identity to their login account.

RESULTS

Recognition Performance:
100% recognition accuracy was achieved across all test attempts with no false positives recorded. Recognition was instant, with both speed and accuracy improving directly with the size of the training dataset.

Liveness Detection:
Blink based liveness detection reliably blocked all photo spoofing attempts during testing. Detection was quick, with response time also benefiting from a larger training dataset.

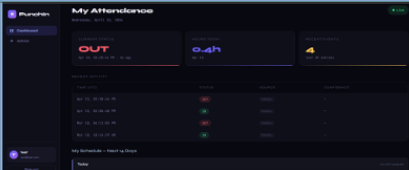
Confidence Scores:
Face distance scores were consistently below 0.65 across all successful recognitions, with scores decreasing as more training images were added, confirming that training data quantity is the primary driver of system performance.

API Response Time:
The FastAPI backend responded to all clock events instantly on the local network, with no noticeable delay observed during testing.

APPLICATIONS

- Small to medium workplace attendance tracking
- Replacement for RFID and PIN systems
- Shift based environments such as retail, hospitality, healthcare
- Secure areas requiring liveness verified access logging

DASHBOARD SCREENSHOT



CONCLUSIONS

The project successfully demonstrates a production ready biometric attendance platform. Face recognition combined with liveness detection provides a secure and contactless clock in/out mechanism. The full-stack architecture from embedded Pi hardware through to a responsive webpage shows the integration of multiple disciplines including backend API design, database management, and frontend development.

GITHUB – RASPBERRY PI



GITHUB – REACT WEBPAGE



3 Introduction

Attendance management is a fundamental requirement in any workplace environment. Punch-in methods such as pin, password or RFID systems are prone to problems such as lost or forgetting RFID card or forgetting a pin and password. For me the main issue I found was having someone else such as a colleague to clock you in or out when you are not physically in the premises. My project addresses all the issues I mentioned by designing and implementing a facial recognition system that identifies employees, eliminating any need for physical interaction with pins, passwords or RFID cards.

My motivation for this project arose from having come up with this idea for a project, the idea of being able to integrate some machine learning with face recognition on my project with some low-cost hardware was a big factor in motivation. I chose to use the Raspberry Pi 4 due to its balance of processing capabilities and cost. Along with choosing the Raspberry Pi 4, I chose to use the Raspberry Pi Camera Module 2 as it was compatible with the board, and it was also capable of carrying out the facial recognition for the project.

The scope of the project covers the full stack of the system with the hardware and facial recognition running on the Raspberry Pi, the REST API backend that processes and stores attendance events, and the webpage that presents data to both employees and administrators. Security was considered throughout the project with me adding in a liveness detection on the facial recognition to prevent spoofing attacks. I also used Firebase Authentication to secure the webpage by letting it handle all the sensitive data with employee and admin passwords as it would not have been practical to store it on the local SQLite database on the Raspberry Pi.

4 Background

4.1 Facial Recognition

Facial recognition is a biometric identification that uses computational methods to identify individuals. In my project it operates in three stages: face detection, feature extraction, and identifying.

I used the `face_recognition` Python library, which is built on `dlib`'s deep learning face recognition. The library generates 128-dimensional face encodings which is a numerical vector representing the unique features of a face. The recognition is performed by computing the Euclidean distance between an unknown encoding and the known encodings in the database. The distance below my defined threshold indicates a match [1].

The face detection is performed using the Histogram of Oriented Gradients (HOG) method, which is suited to real time applications on limited hardware such as the Raspberry Pi [2].

4.2 Liveness Detection

A significant vulnerability I noticed in my facial recognition system was it being susceptible to spoofing, where I could hold up a picture of a user on my phone and the system would not be able to tell the difference between them. To combat this issue, I added a liveness detection in the form of blink checking to the facial recognition system.

The live detection is implemented using Eye Aspect Ratio (EAR) analysis. The EAR is calculated from six facial landmark points around each eye, as defined by the 68-point `dlib` facial landmark predictor. The ratio drops significantly during a blink. By monitoring the ratio across the frames, the system can detect a natural blink and use it as evidence of liveness before triggering the clock event [3].

4.3 FastAPI

FastAPI is a modern, high performance Python web framework for API's. It uses Python type hints for automatic validation via `Pydantic` and provides native supports for asynchronous request handling and Server Sent Events. I chose to use it for its performance and automatic documentation [4].

4.4 Firebase Authentication

Firebase Authentication is a cloud-based identity platform provided by Google. My project uses Firebase email/password authentication. The system verifies the Firebase ID token (JWTs) on the Raspberry Pi using the Firebase Admin SDK, without making external network calls for each verification [5].

4.5 React and Next.js

React is a JavaScript library that is used to build user interfaces. Next.js is a React framework that adds file routing and production optimisations. The Smart Punch-In uses Next.js Router with client-side rendering for all dashboard components, since data is fetched dynamically from the Pi at runtime [6].

4.6 SQLite

SQLite is a lightweight, serverless relational database engine that stores all data in a single file. I chose it because it runs directly on the Raspberry Pi without requiring a separate database server which means my entire system is self-contained [7].

4.7 Server Sent Events

Server Sent Events (SSE) is a web technology that allows a server to push data to a browser over a persistent HTTP connection. Although SSE is one directional only, I chose to use it as I only need to push updates from the server to the client. I compared it to Polling, which is what I had initially, and SSE stood out to me as the better choice than Polling.

5 Project Architecture

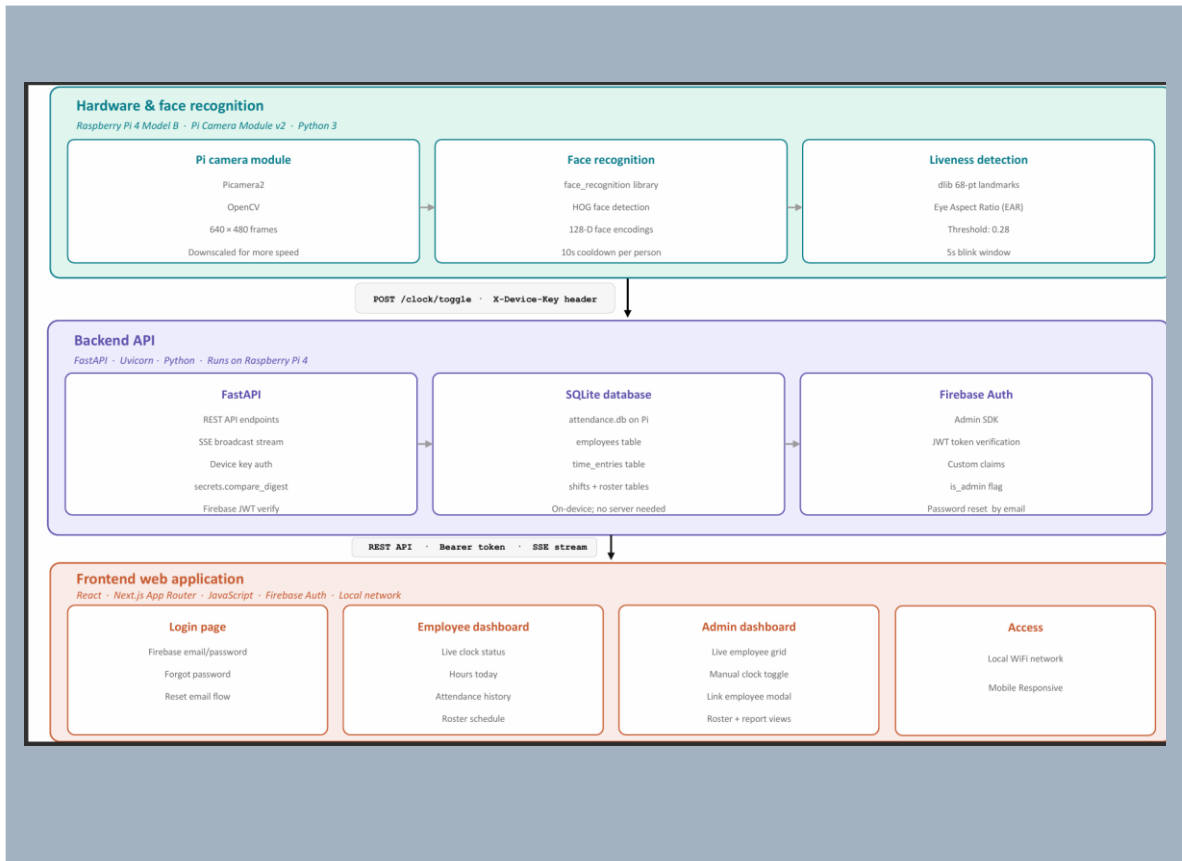


Figure 5-1 Architecture Diagram

6 Project Plan

Throughout the course of the academic year, I split the project into 5 phases, which I worked on throughout the year

6.1 Research

In this phase the objective for me was picking out a project idea and once I had the idea finalised, I carried out background research to see if the idea is something that can be done and what hardware I will need for it. The hardware I needed was the Raspberry Pi 4 and the Raspberry Pi camera module. I also done research on using facial recognition on the Raspberry Pi and that's where I learned about the face_recognition, dlib and OpenCV libraries and found out how they can be used in my project.

6.2 Hardware Setup

Once the hardware I ordered arrived, I set up the configuration of the Raspberry Pi by installing the suitable OS for my project as well creating a virtual environment for my project and then installing the libraries needed for the facial recognition. I also set up the Python environment, making sure the Python version I was using was compatible with the libraries installed. The Pi camera was also set up and with everything set up, I ran a facial recognition demo to make sure everything was working together before making a start on the backend side of the project.

6.3 Backend Development

For the backend development I implemented the Fast Api application, SQLite schema design and setting the admin to the designated user on the database, the REST API endpoints, and SSE broadcasts. I integrated Firebase Authentication to handle all the authentication for the frontend side of it with the login for the webpage.

6.4 Frontend Development

Implementation of Next.js webpage with employee and admin interfaces, Firebase login, manual punch in and out on admin interface, roster system and attendance report. The webpage is also mobile responsive where it has been modified and resized to be presentable when viewing on a phone.

6.5 Integration & Testing

Integrating both the backend and frontend, testing all the components to make sure that both sides of the project are working correctly. Testing the liveness detection, calibrating the EAR threshold for the users, mobile responsiveness making sure the phone can access the webpage with no issues or errors, and final testing off the whole project start to finish to make sure it works as planned.

7 Implementation

7.1 Face Recognition Pipeline

The face recognition pipeline is implemented in my `face_rec_test.py` and runs as a continuous loop on the Raspberry Pi. Each iteration captures a frame from the Pi camera, I downscaled it 6 times (`CV_SCALER = 6`) for efficiency and passes it to the `face_recognition` library for the face detection and encoding. By processing every second frame (`PROCESS_EVERY = 2`) it reduces the lag I initially had on the camera, and it improves the responsiveness.

The figure below shows the core recognition logic by comparing the detected face encodings against the known encodings and identifying the best match.



```

127
128     if known_face_encodings:
129         distances = face_recognition.face_distance(known_face_encodings, face_encoding)
130         best_idx = int(np.argmin(distances))
131         matches = face_recognition.compare_faces(known_face_encodings, face_encoding)
132
133     if matches[best_idx]:
134         name = known_face_names[best_idx]
135         confidence = float(distances[best_idx])
136         now = time.time()
137

```

Shell %
Python 3.11.2, 32-bit (/home/chuka/face_rec/bin/python3)
>>>

The training is done using `model_training.py` which processes all images in the dataset folder and captures all the face encodings to `encodings.pickle`. The recognition accuracy improves significantly with a larger training dataset. When I was testing, it showed that an average of 10-15 training images per person produces reliable results. I also implemented a 10 second cooldown after each clock event to prevent a spam of clock events.

7.2 Liveness Detection

I implemented the liveness detection using the dlib 68-point facial landmark predictor. When a face is recognised, the system enters a 5 second liveness window during which it monitors the Eye Aspect Ratio (EAR) across the consecutive frames. The EAR is calculated using the following formula:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

Where p_1 to p_6 are the six dlib landmark points surrounding the eye. The code below implements the EAR calculation:

```

62
63 # — Helpers —————
64
65 def eye_aspect_ratio(landmarks, eye_indices):
66     pts = np.array([(landmarks.part(i).x, landmarks.part(i).y) for i in eye_indices])
67     A = distance.euclidean(pts[1], pts[5])
68     B = distance.euclidean(pts[2], pts[4])
69     C = distance.euclidean(pts[0], pts[3])
70     return (A + B) / (2.0 * C)
71
72 def toggle_clock(name, confidence=None):
73     True

```

Shell 

Python 3.11.2, 32-bit (/home/chuka/face_rec/bin/python3)

>>>

The threshold was calibrated by logging the live EAR values during testing. Open eye EAR had an average of approximately 0.39, while the blink values ranged from 0.165 to 0.198. The threshold of 0.28 was set at the midpoint, making it a reliable gap between the close and open eye range. The clock event is only triggered after the EAR drops below the set threshold for at

least one consecutive frame, then it rises again confirming a complete blink rather than an eye closing gradually.

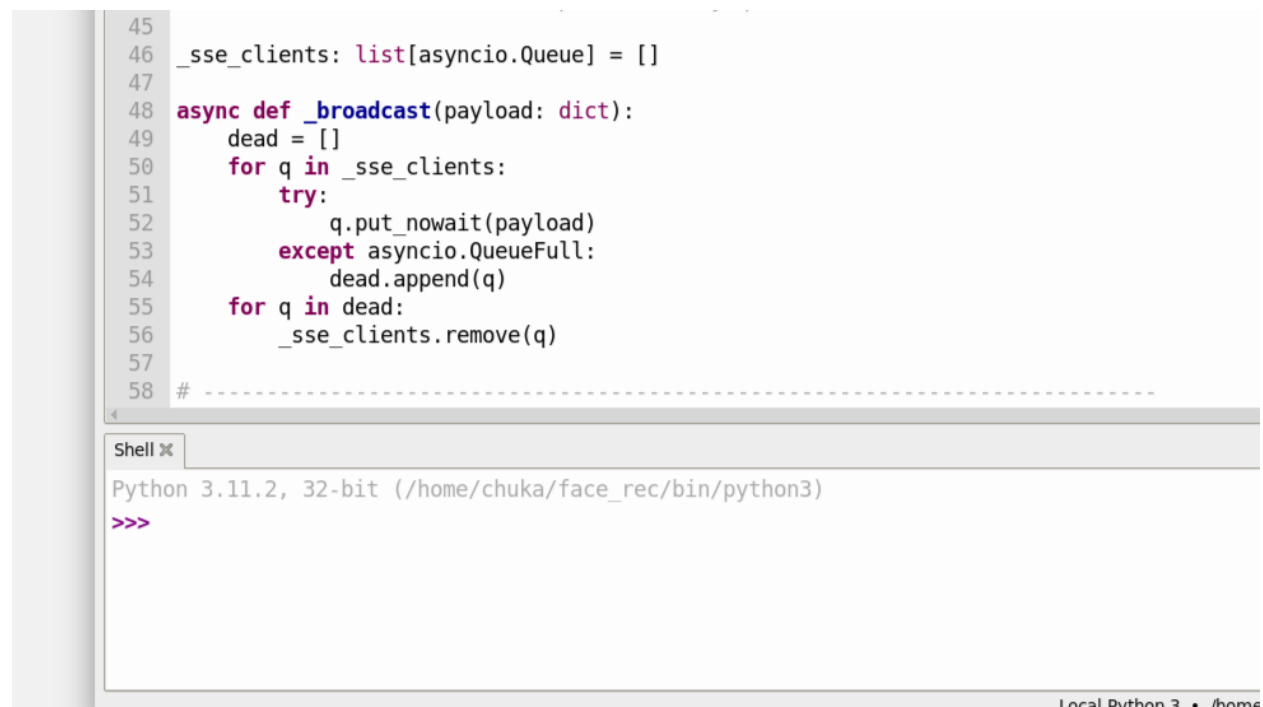
7.3 FastAPI Backend

The Fast API in main.py is structured around three categories of endpoints, each one has its own authentication mechanism:

- Employee endpoints: require a valid Firebase ID token in the authorisation header
- Device endpoint(/clock/toggle): requires an X-Device key header matching the DEVICE_API_KEY
- Admin endpoints: requires both Firebase authentication and is_admin=1 in the database table.

The device API key is verified using Python's `secrets.compare_digest()` functions, which performs a constant time string comparison. This prevents timing attacks, where an attacker could measure response times to deduce the key character by character.

the SSE broadcast shows that each connected client registers an `asyncio.Queue`, and the clock events are pushed to all queues simultaneously:



```

45
46 _sse_clients: list[asyncio.Queue] = []
47
48 async def _broadcast(payload: dict):
49     dead = []
50     for q in _sse_clients:
51         try:
52             q.put_nowait(payload)
53         except asyncio.QueueFull:
54             dead.append(q)
55     for q in dead:
56         _sse_clients.remove(q)
57
58 # -----

```

Shell x

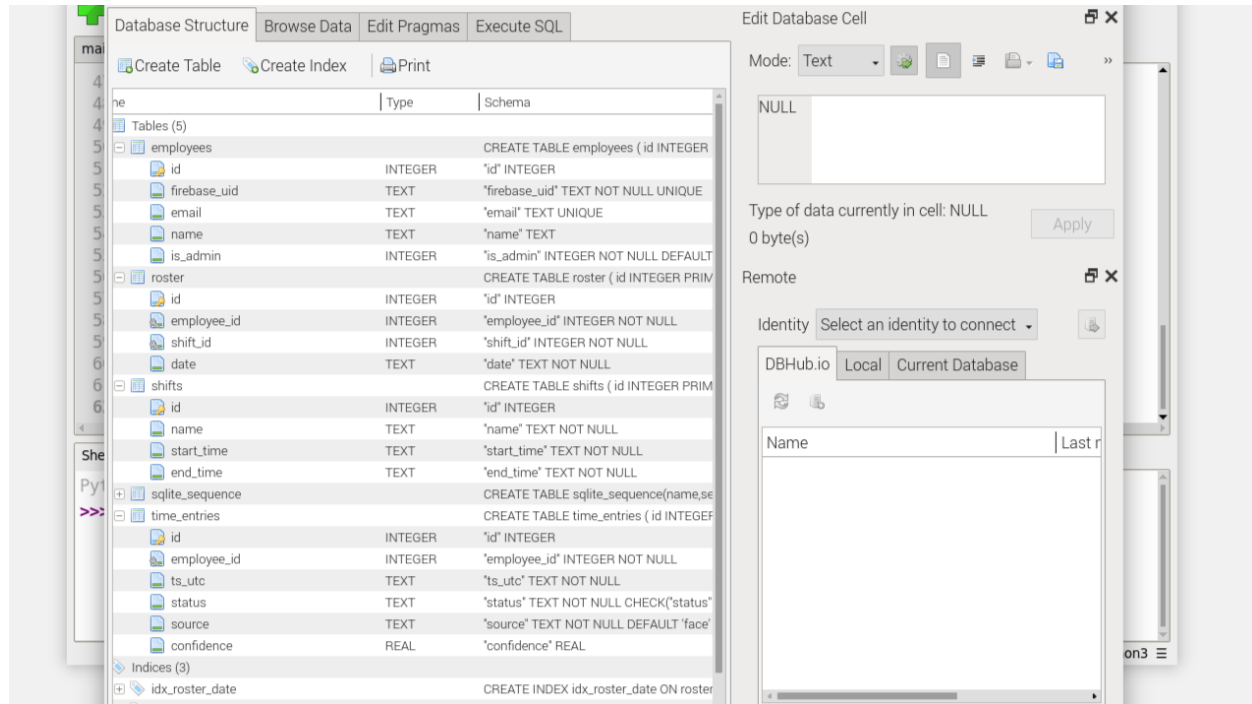
Python 3.11.2, 32-bit (/home/chuka/face_rec/bin/python3)

>>>

Local Python 3 • /home

7.4 Database Schema

The SQLite uses four tables which are initialised by models.py on startup with safe migration if there are any schema changes:



A key design decision I made was the three step Firebase to employee linking chain. The Raspberry Pi creates records by name when a face is first recognised, these records initially have no firebase UID or email. When the admin links a name to an email on the admin interface, the backend creates a Firebase account, updates the existing employee record with the Firebase UID and email, and sends a password invite. This preserves the existing data from before the account was linked.

7.5 Shift Roster System

The shift roster system allows the admins to create reusable shifts templates and assign them to employees daily. The shift templates are stored in the shifts table with a name, start and end time. The admin roster page presents the admin with a weekly grid with all the employees shift for the week.

The backend GET/admin/roster endpoint returns all the employees, all shifts templates and all the roster entries for that week in a single response.

When a cell is clicked, a popover appears allowing the admin to select a shift from the available templates or clear the existing shift. On the employee side on the webpage, the GET/me/roster endpoint returns the employee's schedule for the next 14 days, which is displayed as a panel on their dashboard.

7.6 Attendance Report

The attendance report compares actual clock in and out times against scheduled shifts to identify punctuality issues. The GET/admin/attendance-report endpoint accepts a target date, fetches all roster entries for that date, and for each scheduled employee retrieves their first clock in and last clock out from the time_entries table.

Clock times are compared against shift start and end times with a configurable grace period that I set to 10 minutes (GRACE_MINUTES=10). The system generates the following flags for each employee:

- late_in: first clock in was more than 10 minutes after shift start
- early_out: last clock out was more than 10 minutes before shift end
- no_show: no clock in recorded for the day
- no_clock_out: clocked in but no clock out recorded

the report page displays a summary strip showing totals for each flag category, a filter table with colour coded statuses, and the number of minutes late or early annotated directly on the clock times.

7.7 Frontend

The Next.js frontend uses the App router. All pages use client side since data is fetched dynamically from the Raspberry Pi API. For styling I use CSS class names within the inline style tags in each component, with CSS custom properties defining a consistent dark theme across all pages.

The employee dashboard displays the current clock status, hours worked today, recent attendance history, and upcoming shift schedule. The admin interface provides a live employee status grid updated through SSE, with each employee's status showed. This employee status page on the admin interface shows the hours, a manual clock toggle button and a full clock history on any selected employee.

To make the frontend mobile friendly I had to add a media query where it would make the original sidebar hidden as it takes up too much space when viewing on a phone. With this change I moved the sidebar from the left side of the screen to it being at the top of the screen in a more compact way.

The SSE subscription uses the Fetch API which allows the Firebase ID token to be included in the authorisation header for the SSE connection.

```
const res = await fetch(`${BASE}/events`, {
  headers: { Authorization: `Bearer ${token}` },
  signal: controller.signal,
});

if (!res.ok || !res.body) {
  onError?.(new Error(`SSE connect failed: ${res.status}`));
  return;
}

const reader = res.body.getReader();
```

8 Challenges and solutions

8.1 Wrong Operating system

When setting up the Raspberry Pi, I kept running into multiple errors when installing the libraries needed for the project. The errors were all similar saying that the version of Python installed are too new for the libraries. After a while of trying to revert to an older version, it still did not fix the errors until I decided to completely wipe the Raspberry Pi and start from the beginning. When setting the Raspberry Pi up it gave me three operating systems to choose from, I done quick research on all three to find which version of python is used, and once I found the correct operating system (Bookworm) which uses Python 3.11 I installed it and ran into no issues with the libraries. Initially I set up the Raspberry Pi using operating system Trixie which uses Python 3.13.5 and was nit compatible with the libraries.

8.2 SQLite Database Locking

A “database is locked” error was encountered in the employee account linking endpoint. The root cause was that the endpoint opened a SQLite connection, read employee data, close the connection then create a Firebase account and then open a second connection to write the updated record. The gap between the two connections caused the “database is locked” message. To fix this I changed the endpoint to use a single connection for the entire operation. I wrapped it in a try/finally to make sure the connection was always closed after the operation is completed.

8.3 Liveness Detection EAR Calibration

When setting up the EAR threshold values, I wasn't too sure what the threshold should be. Initially I had it set at 0.15 and it didn't perform reliably in the test, to fox this I wrote a script that would display the EAR values in the terminal, allowing me to observe directly of normal eye activity and deliberate blinking. With the script it helped me to measure the difference between the open eye and the blinking values. I set the new and current threshold at 0.29 to allow for some headroom in both directions and the blink detection in the program has since performed at a more consistent rate.

9 Results

The complete system was tested across multiple sessions with two users under standard indoor lighting conditions. The following results were recorded:

Recognition accuracy	100% across all test attempts
False positives	None recorded during testing
Photo spoofing attempts blocked	The spoofing attempts blocked any attempts with a photo of the user, however a video of a user blinking still bypassed the detection.
Average recognition speed	Near instantaneous upon entering the frame
API response time (clock event)	Under 1 second in all cases on local network
Confidence score	Consistently below 0.65; lower scores with larger training dataset
Liveness window	5 second per recognition
Cooldown per person	10 seconds

The recognition accuracy and speed both showed a direct dependency on the training size. With few than 5 training images the system might take longer than usual to recognise a face. With 10 or more images, the recognition was consistent and reliable across all test conditions. This relationship is consistent with the documentation of the `face_recognition` library, which relies on sufficient training variation to build robust encodings.

Liveness detection performed well against all photo attempts, but when it came to videos of a user blinking It bypassed the detection. The liveness detection was very consistent with detecting static images and never accepted any static image as a live face.

The attendance report correctly flagged late arrivals and early departures when the test clock events were outside the scheduled shift times. The 10-minute grace period worked as expected, with events within the grace window were recorded as on time.

10 Ethics

The face encodings are biometric data, unlike passwords it can't be changed if compromised. In this system, face encodings are securely stored securely on the Raspberry Pi, and the data is not transferred to an external server. This minimises the risk of a data breach compared to a cloud-based service. In a production deployment, physical access to the Raspberry Pi and its storage would need to be secured.

11 Conclusion

This project successfully demonstrates the design and implementation of a fully functional end to end facial recognition attendance system. The system achieves the primary goal of automating employee punch in and out through contactless biometric identification, with liveness detection providing a meaningful defence against all photo spoofing attempts.

The full stack architecture combines embedded hardware, REST API backend, real time events, and a responsive webpage demonstrates the integration of multiple engineering disciplines including computer vision, backend API design, database management, web development and security engineering.

There are multiple key outcomes of this project such as a working facial recognition pipeline running in real time on the Raspberry Pi 4, Fast Api backend with Firebase Authentication, SQLite storage, SSE live updates, a React webpage allowing employees and admins to view shift roster and attendance reporting system and having the webpage page mobile friendly.

Future development for this project that I would implement would be to implement depth-based detection for stronger anti spoofing as security is a very important factor when it comes to dealing with sensitive data.

12 Appendix

12.1 Project File Structure

File	Description
main.py	Fast API application: all API routes, SSE broadcast, Firebase linking logic
models.py	SQLite schema initialisation
db.py	Database connection helper with row factory
auth_firebase.py	Firebase token verification Fast Api dependency
auth_device.py	Device API key verification
face_rec_test.py	Live face recognition loop with liveness detection
model_training.py	Encodes training images into encodings.pickle
image_capture.py	Photo capture to register new employees
src/app/login/page.js	Firebase login page with forgot password modal
src/app/dashboard/page.js	Employee attendance dashboard with SSE updates
src/app/admin/page.js	Admin overview with employee grid and link modal
src/app/admin/roster/page.js	Weekly shift roster management
src/app/admin/report/page.js	Attendance punctuality report
src/components/RosterPanel.js	Employee upcoming 14 day schedule
src/lib/api.js	API helpers and SSE subscription

12.2 API Endpoint Reference

Endpoint	Description
GET /me	Get or create employee record for authenticated user
GET /me/status	Current IN/OUT status and last event timestamp
GET /me/history	Gets clock event history
GET /me/hours	Total hours worked for a given date
GET /me/roster	Upcoming shift schedule for the next 14 days

GET /events	SSE stream broadcasts all clock events to the connected clients
POST /clock/toggle	Raspberry Pi clock event trigger
GET /admin/summary	All employees with live status and hours today
GET /admin/employees/unlinked	Employees with no email or Firebase account linked
POST /admin/employees/link	Link employee record to Firebase account, sends invite
POST /admin/employees/{id}/clock	Admin manual clock toggle for any employee
GET /admin/roster	Full weekly roster grid for all employees
POST /admin/roster	Assign or update shifts for employees
DELETE /admin/roster/{id}	Remove a roster entry
GET /admin/shifts	List all shift templates
POST /admin/shifts	Create a new shift template
GET /admin/attendance-report	Punctuality report comparing clock times to shifts

13 References

- [1] A. Geitgey, "face_recognition" GitHub. [Online]. Available: https://github.com/ageitgey/face_recognition.
- [2] GeeksforGeeks, "Histogram of Oriented Gradients," *GeeksforGeeks*, Jul. 23, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-vision/histogram-of-oriented-gradients/>.
- [3] T. Soukupová and J. Čech, "Real-Time Eye Blink Detection using Facial Landmarks," in *Proc. 21st Computer Vision Winter Workshop (CVWW)*, Rimske Toplice, Slovenia, Feb. 2016, pp. 1–8. [Online]. Available: <https://vision.fe.uni-lj.si/cvww2016/proceedings/papers/05.pdf>.
- [4] S. Ramirez, "FastAPI," [Online]. Available: <https://fastapi.tiangolo.com>.
- [5] Google, "Firebase Authentication," [Online]. Available: <https://firebase.google.com/docs/auth>.
- [6] Vercel, "Next.js Documentation," [Online]. Available: <https://nextjs.org/docs>.
- [7] D. R. Hipp, "SQLite," [Online]. Available: <https://www.sqlite.org>.
- [8] Tim, "Face Recognition With Raspberry Pi and OpenCV," *Core Electronics*, Oct. 16, 2024. [Online]. Available: <https://core-electronics.com.au/guides/face-identify-raspberry-pi/>.
- [9] D. E. King, "dlib," *Python Package Index (PyPI)*, version 19.24.6. [Online]. Available: <https://pypi.org/project/dlib/>.